深度学习
第 8 章　卷积神经网络（CNN）

——从卷积运算到 LeNet，理解 CNN 的核心设计哲学

2026 年 3 月 16 日

本章题目导览：你将挑战哪些核心问题？

CNN 的核心优势在于**局部连接**与**参数共享**，本章 6 道题从多个维度全面覆盖第 8 章知识点：

- **Q8-01（填空题）**：卷积基本术语与特性——池化层有无可学习参数？通道数如何变化？
- **Q8-02（计算题）**：卷积输出形状与参数量计算——给定 I, K, P, S，算出特征图大小和权重数量。
- **Q8-03（辨析题）**：填充的作用与 SAME/VALID 两种模式——为什么奇数卷积核才能做 SAME 填充？
- **Q8-04（计算题）**：3 维卷积的通道计算——多输入通道、多输出通道时，参数量如何随通道数平方增长？
- **Q8-05（对比分析题）**：经典网络架构横向对比——AlexNet、VGG、GoogLeNet、ResNet 各自解决了什么问题？
- **Q8-06（代码阅读题）**：LeNet 代码追踪——逐层追踪 Fashion-MNIST 案例中特征图的形状变化。

1　第 8 章　卷积神经网络（CNN）

Q8-01 卷积神经网络基本概念填空

题目描述：

根据第 8 章的内容，填写以下关于 CNN 核心概念的空白。

(1) CNN 中新出现了两种层：_____ 和 _____，前者用于提取局部特征，后者用于降维和增强鲁棒性。

(2) 与全连接层相比，卷积层的两大核心优势是：_____ 和 _____。

(3) 池化层_____（有/没有）可学习的参数，且经过池化后数据的_____不会发生变化。

(4) 卷积运算中，输入数据与卷积核进行逐元素相乘后_____，该操作也称为_____。

(5) 在 CNN 中，每个卷积层的输入和输出数据也被称为_____（英文：_____）。

(6) Max 池化选取窗口内的_____，Average 池化选取窗口内的_____。

来源溯源：[文档第 8 章 8.1 CNN 概述 / 8.2 卷积层 / 8.3 池化层]

小白必读：卷积层 vs 全连接层，核心差异在哪？

- **全连接层的问题：** 处理 $224 \times 224 \times 3$ 的图片时需要将其**拉平为一维向量**（约 15 万个数），这会丢失空间位置信息——左边的像素和右边的像素被一视同仁地放进同一个向量，原本的”左右关系”荡然无存。
- **卷积层的解决方案：** 保持数据的三维形状（通道、高、宽）原封不动地传递。卷积核只看输入的一小块区域（局部连接），同一个卷积核在整张图上用同一套权重扫描（参数共享）。这两点使得卷积层既保留了空间信息，又大幅减少了参数量。
- **特征图 vs 输入图像：** 卷积层的输入输出都叫”特征图”（Feature Map）。原始图像是第一张特征图，经过第一层卷积后得到第二批特征图（每个卷积核产生一张），以此类推。越深的层，特征图表示的语义越抽象：浅层是边缘、纹理，深层是部件、整体概念。

参考答案与详细解析

(1) **卷积层**（Convolution 层）和**池化层**（Pooling 层）。

(2) **局部连接**（每个输出只和输入的一小块区域相连）和**参数共享**（同一卷积核在整图上共用一套权重）。

(3) 池化层**没有**可学习的参数；经过池化后数据的**通道数**不会发生变化（池化按通道独立进行）。

(4) 逐元素相乘后**求和**；该操作也称为**乘积累加运算（内积）**。

(5) 输入和输出数据称为**特征图**（英文：**Feature Map**）。

(6) Max 池化选取窗口内的**最大值**；Average 池化选取窗口内的**平均值**。

核心记忆点：池化层的三大特性

1. **无参数**：池化操作（取最大值/均值）是固定规则，没有需要梯度更新的权重。
2. **通道数不变**：C 个通道输入，C 个通道输出，每个通道独立池化。
3. **鲁棒性**：数据轻微偏移（如猫的位置挪了 1 像素），Max Pooling 大概率返回相同结果——这给 CNN 带来了**平移不变性**。

现实应用场景

为什么医学影像 AI 首选 CNN？

CT 扫描、X 光片等医学图像的核心特征（肿瘤边缘、钙化点、病灶纹理）往往是**局部的**、**与位置无关的**——无论肿瘤长在肺的左侧还是右侧，它的纹理特征是相似的。CNN 的参数共享天然适配这一特性：同一个”边缘检测器”卷积核可以在整张 CT 图上复用，而不需要为每个位置单独训练一个检测器。全连接网络在 512×512 的医学影像上参数量可达亿级，CNN 则可将参数量压缩到百万级以内，大幅降低了对标注数据量的需求。

Q8-02　卷积输出形状与参数量计算

题目描述：

标准二维卷积的输出尺寸公式（以高度方向为例）为：

$$H_{out} = \left\lfloor \frac{H_{in} + 2P - K}{S} \right\rfloor + 1 \tag{1}$$

其中 H_{in} 为输入高度，K 为卷积核大小，P 为填充，S 为步幅，$\lfloor \cdot \rfloor$ 为向下取整。

条件一（计算输出形状）：

输入特征图形状为 ($C_{in} = 3$，$H = 100$，$W = 100$)，卷积参数：$K = 9$，$S = 3$，$P = 0$，$C_{out} = 3$。

问题：

(1) 计算该卷积层输出特征图的形状 (C_{out}，H_{out}，W_{out})。

(2) 该卷积层共有多少可训练参数（含偏置）?

(3) 若步幅改为 $S = 1$，填充改为 $P = 4$，其他不变，输出特征图的高度 H_{out} 是多少?

来源溯源：[文档第 8 章 8.2.1 卷积运算 / 8.2.3 步幅 / 8.2.5 API 使用]

小白必读：输出尺寸公式怎么理解？

- **直觉推导：** 卷积核从输入左边第一个合法位置出发，每次移动 S 步，直到无法再移动为止。可以放置的位置数 = 输出尺寸。
- **公式拆解：** $H_{in}+2P$ 是填充后的输入高度；减去 K 是因为第一次放下卷积核就占了 K 个位置，剩余 $(H_{in}+2P-K)$ 步可以移动，除以步幅 S 得到移动次数，再加 1（算上起始位置）即为输出尺寸。
- **参数量计算：** 一个卷积核的参数数量 $=K\times K\times C_{in}$（宽 × 高 × 输入通道数），共有 C_{out} 个卷积核，再加每个输出通道一个偏置：

$$\text{总参数量}=C_{out}\times(K\times K\times C_{in}+1) \tag{2}$$

- **向下取整的含义：** 当输入尺寸除不尽时，PyTorch 默认丢弃最右/最下不完整的区域，输出取整数。

生活比喻：用尺子量布

把输入特征图想象成一块布，卷积核是一把尺子（长度为 K）：

- **填充** P：在布的两端各接上 P 厘米的空白边，让尺子能从最边缘开始量。
- **步幅** S：每量完一次，把尺子向右移动 S 厘米再量一次。步幅越大，量的次数越少，输出就越小。
- **向下取整**：布的末端剩余的长度不够一把尺子，就放弃这段——这正是向下取整的含义。
- **参数共享的妙处**：整块布（整张特征图）上，用的始终是**同一把尺子**，尺子的”刻度”（权重）全程不变。不同卷积核就是刻度不同的尺子，每把尺子负责量出一张特征图。

参考答案与详细解析

(1) 计算输出特征图形状：

代入公式（高度和宽度对称，计算相同）：

$$H_{out}=W_{out}=\left\lfloor\frac{100+2\times0-9}{3}\right\rfloor+1=\left\lfloor\frac{91}{3}\right\rfloor+1=30+1=\mathbf{31} \tag{3}$$

输出特征图形状为：$\boxed{(C_{out}=3,\ H_{out}=31,\ W_{out}=31)}$

(2) 计算可训练参数量：

每个卷积核参数数量（含偏置）$=K\times K\times C_{in}+1=9\times9\times3+1=243+1=244$

共有 $C_{out}=3$ 个卷积核，因此总参数量为：

$$\text{总参数量}=3\times244=\mathbf{732} \tag{4}$$

(3) 修改参数后的输出高度（$S=1$，$P=4$）：

$$H_{out}=\left\lfloor\frac{100+2\times4-9}{1}\right\rfloor+1=\left\lfloor\frac{99}{1}\right\rfloor+1=99+1=\mathbf{100} \tag{5}$$

结论：$P=4$，$K=9$，$S=1$ 时输出尺寸与输入相同！

一般地，当步幅 $S=1$ 时，令 $P=\lfloor K/2\rfloor$（对奇数核恰好等于 $(K-1)/2$），可使输出尺寸等于输入尺寸（SAME 填充）。$K=9$ 时 $P=4$，恰好满足此条件，所以 $H_{out}=H_{in}=100$。

对应 API 代码验证：

```python
import torch
import torch.nn as nn

# 题目条件一
conv1 = nn.Conv2d(in_channels=3, out_channels=3, kernel_size=9,
                  stride=3, padding=0)
x = torch.randn(1, 3, 100, 100)
print(conv1(x).shape)   # torch.Size([1, 3, 31, 31])  -- 验证(1)

# 修改后的条件
conv2 = nn.Conv2d(in_channels=3, out_channels=3, kernel_size=9,
                  stride=1, padding=4)
print(conv2(x).shape)   # torch.Size([1, 3, 100, 100]) -- 验证(3)

```

现实应用场景

AlexNet 第一层卷积（2012 ImageNet 冠军）：

AlexNet 接受 $227 \times 227 \times 3$ 的彩色图像，第一层卷积参数为 $K = 11$，$S = 4$，$P = 0$，$C_{out} = 96$，输出特征图形状为 $(96, 55, 55)$，参数量 $= 96 \times (11 \times 11 \times 3 + 1) = 34{,}944$ 个。若用全连接层处理同一输入，光这一层就需要 $227 \times 227 \times 3 \times 55 \times 55 \times 96 \approx 450$ 亿个参数——这正是 CNN 参数效率优势的量化体现。

Q8-03 填充的作用与 SAME/VALID 模式辨析

题目描述：

文档中提到，填充的主要目的是”调整输出数据的形状大小，避免多次卷积后数据形状过小”，同时还能”令数据形状在卷积后保持不变”。

问题：

(1) 对形状为 4×4 的输入数据进行幅度为 1 的零填充，填充后的形状是多少？再经过 3×3、步幅为 1 的卷积后，输出形状是多少？说明填充如何使输出与输入形状保持一致。

(2) 不使用填充（$P = 0$）时，对 $H \times W$ 的特征图反复进行 3×3（步幅 1）的卷积，每次卷积后尺寸减少多少？若初始尺寸为 28×28，最多能卷积多少次才不会使特征图尺寸降至 1×1 以下？

(3) 为什么 SAME 填充模式通常要求卷积核大小为**奇数**？如果卷积核大小为偶数（如 $K = 4$），SAME 填充会出现什么问题？

来源溯源：[文档第 8 章 8.2.2 填充]

小白必读：填充为什么几乎总是补 0？

- **零填充（Zero Padding）的含义：**在输入特征图的四周补上值为 0 的像素圈。补 1 圈（$P = 1$）就在上下左右各加一行/列 0，补 2 圈（$P = 2$）则各加两行/列 0。
- **为什么补 0 而不补其他值？**补 0 对卷积结果的影响最小——与 0 相乘等于 0，不会引入额外的"假信息"。
- **边缘像素的不公平性：**不填充时，位于图像边角的像素只被卷积核覆盖 1 次，而中心像素被覆盖 $K \times K$ 次（对 3×3 核就是 9 次）。这导致边缘特征被"欠学习"。填充后，边缘像素同样能被完整的卷积核覆盖多次，地位更公平。
- **SAME vs VALID 模式：**
  - **VALID**（不填充）：$P = 0$，输出尺寸 < 输入尺寸。
  - **SAME**（保持尺寸）：选择合适的 P 使输出 = 输入尺寸（步幅为 1 时）。

生活比喻：拍照时的构图裁剪

把卷积想象成对一张照片做"局部聚焦分析"：

- **不填充（VALID）**：相机只能对准照片的**中心区域**拍摄，每次拍完后照片变小一圈——边缘的风景越来越被切掉，拍得越多，照片越小，最终缩成一个点就再也没法拍了。
- **填充（SAME）**：在照片四周贴上一圈白色边框，让相机也能对准边缘拍摄。贴几圈、卷积核多大，都能算好，保证每次"拍出来的照片"和原来一样大，信息不丢失。
- **奇数核的必要性**：边框要**对称**地贴在四周，所以每边贴的量必须是整数。$K = 3$ 时需要 $P = (3-1)/2 = 1$（每边 1 圈），$K = 5$ 时需要 $P = 2$（每边 2 圈）——奇数核总能整除。$K = 4$ 时需要 $P = (4-1)/2 = 1.5$，但是圈数不能是 0.5——要么上边贴 1 圈下边贴 2 圈（不对称），输出就会出现半像素的错位，这在工程中是不可接受的。

参考答案与详细解析

(1) 零填充后形状变化：

4×4 输入加 $P = 1$ 填充后，每个维度增加 $2P = 2$，形状变为 $\mathbf{6 \times 6}$。

经过 $K = 3$，$S = 1$，$P = 0$ 卷积：

$$H_{out} = \left\lfloor \frac{6 + 0 - 3}{1} \right\rfloor + 1 = 3 + 1 = \mathbf{4} \tag{6}$$

输出形状为 4×4，与输入相同。**这说明 $P = 1$，$K = 3$，$S = 1$ 时，卷积不改变特征图尺寸**。

(2) 不填充时的尺寸衰减：

每次 3×3，$S = 1$，$P = 0$ 的卷积，尺寸减少：

$$\Delta = K - 1 = 3 - 1 = 2 \quad \text{（每次高/宽各减少 2）} \tag{7}$$

初始 28×28，设卷积 n 次后尺寸为 $28 - 2n$，要求 $28 - 2n \geq 2$（至少 2×2 才能再卷一次 3×3）：

$$n \leq \frac{28 - 2}{2} = 13 \tag{8}$$

最多卷积 **13** 次，第 13 次后尺寸为 $28 - 26 = 2$；再卷一次（第 14 次）会因为 $2 < 3 = K$ 而无法进行（或降至无效尺寸）。

(3) 奇数核与 SAME 填充：

SAME 填充要求步幅 $S = 1$ 时 $H_{out} = H_{in}$，代入公式：

$$H_{in} = \left\lfloor \frac{H_{in} + 2P - K}{1} \right\rfloor + 1 \implies 2P = K - 1 \implies P = \frac{K-1}{2} \tag{9}$$

P 必须是**非负整数**，故 $K - 1$ 必须是偶数，即 K 必须是**奇数**。

若 $K = 4$（偶数），则 $P = 3/2 = 1.5$——无法用整数填充实现完美的 SAME 模式。实际工程中只能取 $P = 1$（上下各 1）或在上下两边不对称填充（上 1 下 2），两种方案都会导致输出偏移或非对称，破坏特征图的空间对齐性，给后续的残差连接（形状必须完全匹配）带来麻烦。

总结：填充三大结论

1. 填充幅度 $P = 1$、$K = 3$、$S = 1 \Rightarrow$ 输出尺寸 = 输入尺寸（SAME 模式）
2. 不填充时每次卷积尺寸减少 $K - 1$（3×3 核每次减 2）
3. SAME 填充要求 K 为奇数，这正是 ResNet、VGG 全面采用 3×3 和 1×1 卷积核的重要原因之一

现实应用场景

U-Net 中的填充策略（医学图像分割）：

U-Net 是医学图像分割领域最经典的网络。其”编码器-解码器”结构中，解码器阶段需要将深层特征图**上采样后与编码器的特征图拼接**，这要求两者的高宽完全一致。如果卷积层使用 VALID（无填充），每次卷积都缩小特征图，解码器与编码器的尺寸就对不上，无法拼接。因此 U-Net 中几乎所有卷积层都使用 3×3 核加 $P = 1$ 的 SAME 填充，保证特征图尺寸在编解码两端严格匹配。

Q8-04 三维卷积的通道与参数量计算

题目描述：

真实图像是三维数据（通道数 C_{in}、高度 H、宽度 W）。进行三维卷积时，输入通道数必须与卷积核的通道数相同。使用 C_{out} 个卷积核时，输出通道数为 C_{out}。

题目： Fashion-MNIST 案例中，文档给出了如下模型结构：

- 第 1 层：`Conv2d(in=1, out=6, kernel=5, padding=2)`，输入为 28×28 灰度图
- 第 2 层：`AvgPool2d(kernel=2, stride=2)`
- 第 3 层：`Conv2d(in=6, out=16, kernel=5)`（无填充）
- 第 4 层：`AvgPool2d(kernel=2, stride=2)`
- 第 5 层：`Flatten()`
- 第 6 层：`Linear(400, 120)`

问题：

(1) 逐层追踪特征图的形状，填写下表（批次维度忽略，只写 (C, H, W)）：

层编号	层类型	C	H	W
输入	灰度图像	1	28	28
第 1 层	Conv2d(1,6,K=5,P=2)	—	—	—
第 2 层	AvgPool2d(K=2,S=2)	—	—	—
第 3 层	Conv2d(6,16,K=5,P=0)	—	—	—
第 4 层	AvgPool2d(K=2,S=2)	—	—	—
第 5 层	Flatten	—	—	—

(2) Flatten 后的向量长度是多少？与 `Linear(400, 120)` 的输入维度 400 是否吻合？

(3) 计算第 3 层卷积（`Conv2d(6,16,5)`）的参数量（含偏置）。

来源溯源：[文档第 8 章 8.2.4 三维数据的卷积运算 / 8.5.2 搭建模型]

小白必读：通道数在 CNN 中是怎么变化的？

- **卷积层改变通道数：** 输入 C_{in} 个通道，使用 C_{out} 个卷积核，输出 C_{out} 个通道。通道数的变化完全由卷积核数量决定，与 K、P、S 无关。
- **池化层不改变通道数：** 2×2 池化对每个通道**独立**地缩小空间尺寸（高宽各变为原来的一半），通道数 C 保持不变。
- **参数量随通道数平方增长：** 一个卷积层的参数 $= C_{out} \times (K^2 \times C_{in} + 1)$。若 C_{in} 和 C_{out} 都翻倍，参数量约翻 4 倍——这是 CNN 深层参数量急剧增大的根本原因。
- **Flatten 的作用：** 将 (C, H, W) 的三维张量压平为 $C \times H \times W$ 的一维向量，以便输入全连接层。此时必须保证前面各层的输出尺寸使得 $C \times H \times W$ 恰好等于全连接层的输入维度。

参考答案与详细解析

(1) 逐层特征图形状追踪：

- **第 1 层 Conv2d(1,6,K=5,P=2,S=1)**：$H_{out} = \lfloor (28+4-5)/1 \rfloor + 1 = 28$，通道变 6。形状：$(6, 28, 28)$
- **第 2 层 AvgPool2d(K=2,S=2)**：高宽各缩小一半，通道不变。形状：$(6, 14, 14)$
- **第 3 层 Conv2d(6,16,K=5,P=0,S=1)**：$H_{out} = \lfloor (14+0-5)/1 \rfloor + 1 = 10$，通道变 16。形状：$(16, 10, 10)$
- **第 4 层 AvgPool2d(K=2,S=2)**：高宽各缩小一半，通道不变。形状：$(16, 5, 5)$

层编号	层类型	C	H	W
输入	灰度图像	1	28	28
第 1 层	Conv2d(1,6,K=5,P=2)	**6**	**28**	**28**
第 2 层	AvgPool2d(K=2,S=2)	**6**	**14**	**14**
第 3 层	Conv2d(6,16,K=5,P=0)	**16**	**10**	**10**
第 4 层	AvgPool2d(K=2,S=2)	**16**	**5**	**5**
第 5 层	Flatten	$16 \times 5 \times 5 = \mathbf{400}$		

(2) Flatten 后向量长度：

$$16 \times 5 \times 5 = \mathbf{400} \tag{10}$$

与 `Linear(400, 120)` 的输入维度完全吻合！这验证了模型结构设计的正确性。

(3) 第 3 层卷积参数量：

$C_{in} = 6$，$C_{out} = 16$，$K = 5$

$$\text{参数量} = C_{out} \times (K^2 \times C_{in} + 1) = 16 \times (25 \times 6 + 1) = 16 \times 151 = \mathbf{2{,}416} \tag{11}$$

这就是 LeNet-5！

本题追踪的模型正是 1998 年 Yann LeCun 提出的 **LeNet-5**，是深度学习史上第一个实用的卷积神经网络，最初用于手写数字识别（MNIST）。这里将其应用到 Fashion-MNIST 上，原汁原味地重现了 CNN 的开山之作。LeNet-5 的整体参数量约 6 万个——这个数字放到今天几乎微不足道，但在 1998 年是巨大的工程突破。

现实应用场景

为什么现代网络通道数要从 64 起步，越来越大？

LeNet-5 的通道数只有 6 和 16。而现代网络（如 ResNet-50）从 64 通道开始，最深处达到 2048 通道。这是因为：更多的通道意味着网络可以同时学习更多种类的特征（64 个不同的”纹理检测器”、”边缘检测器”……），从而提高模型的表达能力。代价是参数量随通道数平方增长，因此现代网络引入了 1×1 卷积（瓶颈结构）来在增加通道数的同时控制参数量——这是 GoogLeNet Inception 模块的核心设计思想。

Q8-05 深度卷积神经网络经典架构对比

题目描述：

文档第 8.4 节介绍了四个里程碑式的深度卷积神经网络：AlexNet（2012）、VGG（2014）、GoogLeNet（2014）、ResNet（2015）。

问题：

(1) 填写下表，将各网络与其核心创新点对应：

网络名称	核心创新点（从下方选项中选择，每项只用一次）
AlexNet	_____
VGG	_____
GoogLeNet	_____
ResNet	_____

选项：

A. 引入残差连接（跳跃连接），学习残差 $h(x) - x$，解决深度网络梯度消失问题

B. 引入 Inception 结构，横向扩展（多尺度卷积并行），减少参数量

C. 首次证明深度 CNN 在大规模图像分类中的优越性，引发深度学习热潮，使用 ReLU 激活函数和 Dropout

D. 全部使用 3×3 小卷积核堆叠，证明”深度优于单层大核”，结构简洁规整

(2) ResNet 提出的”残差学习”中，网络学习的目标从 $h(x)$ 变为了什么？”快捷结构”（跳跃连接）的数学形式是什么？

(3) VGG 的全称是什么？其 VGG-16 和 VGG-19 中的数字分别指什么？

来源溯源：*[文档第 8 章 8.4 深度卷积神经网络]*

小白必读：为什么网络越来越深，问题却越来越多？

- **直觉上的困惑**：更深的网络学习能力更强，按道理应该越深越好，但实验发现：超过某个深度后，更深的网络在训练集上的误差反而比浅网络更高——这不是过拟合（测试集表现也变差），而是网络**根本没有被训练好**。
- **根本原因：梯度消失**。反向传播时梯度从输出层往输入层传递，每经过一层就要乘以本地梯度。若本地梯度普遍小于 1，连乘 100 层后，浅层的梯度趋近于 0，这些层的参数几乎无法更新——网络越深，浅层”学得越慢”甚至”完全不学”。
- **ResNet 的天才解法**：在每隔几层的地方加一条”高速公路”（跳跃连接），把输入 x 直接加到几层卷积之后的输出上。这条高速公路让梯度可以**绕过卷积层直接传回去**，不管网络多深，浅层总能收到有效的梯度信号。
- **比喻**：就像在多层楼之间加了一部电梯（跳跃连接）——没有电梯时，消息只能一层层往下传，传到底层时已经面目全非；有了电梯，底层随时能直接接收到顶层的原始消息。

参考答案与详细解析

(1) 网络与创新点对应：

网络名称	核心创新点
AlexNet	**C**——首次大规模验证深度 CNN，使用 ReLU + Dropout
VGG	**D**——全用 3×3 小核堆叠，证明深度优于宽度
GoogLeNet	**B**——Inception 结构，多尺度并行卷积
ResNet	**A**——残差连接，解决梯度消失，使超深网络成为可能

(2) ResNet 残差学习：

原始目标是让网络直接学习 $h(x)$（期望输出）。ResNet 改为让网络学习**残差** $F(x) = h(x) - x$，实际输出为：

$$\text{output} = F(x) + x \tag{12}$$

其中 x 通过跳跃连接（恒等映射）直接加到输出上。若最优解接近恒等映射（$h(x) \approx x$），则网络只需学 $F(x) \approx 0$——而让一个网络学" 接近 0 的函数" 远比学" 接近恒等映射的函数" 容易。

(3) VGG 全称与编号含义：

VGG 全称：**Visual Geometry Group**（牛津大学视觉几何组）。

VGG-16 中的 **16** 和 VGG-19 中的 **19** 分别指网络中**含有可学习参数的层数**（卷积层 + 全连接层的总数），分别为 16 层和 19 层。

四大里程碑总结时间线

年份	网络	层数	最大贡献
2012	AlexNet	8	深度学习元年，引爆 CV 革命
2014	VGG	16/19	极简设计哲学，迁移学习骨干
2014	GoogLeNet	22	Inception 模块，横向深度
2015	ResNet	50/101/152	残差连接，深度无上限

现实应用场景

ResNet 至今仍是工业界最常用的骨干网络：

ResNet-50 在 ImageNet 上的 Top-5 准确率达 93.3%，而参数量仅约 2500 万——效率与精度的极佳平衡点。在自动驾驶（目标检测）、医疗影像（病灶分割）、工业质检（缺陷识别）等场景，ResNet 几乎是首选的预训练骨干网络。其残差连接的思想也被后续所有主流架构（DenseNet、EfficientNet、甚至 Transformer 的层归一化）所继承，堪称深度学习历史上影响最深远的单篇论文（引用量超 20 万次）。

Q8-06 代码阅读：Fashion-MNIST 训练流程分析

题目描述：

阅读以下文档中 Fashion-MNIST 案例的训练代码片段，回答问题。

```python
model = nn.Sequential(
    nn.Conv2d(1, 6, kernel_size=5, padding=2),  # 第1层
    nn.Sigmoid(),
    nn.AvgPool2d(kernel_size=2, stride=2),       # 第2层
    nn.Conv2d(6, 16, kernel_size=5),             # 第3层
    nn.Sigmoid(),
    nn.AvgPool2d(kernel_size=2, stride=2),       # 第4层
    nn.Flatten(),                                # 第5层
    nn.Linear(400, 120),
    nn.Sigmoid(),
    nn.Linear(120, 84),
    nn.Sigmoid(),
    nn.Linear(84, 10),                           # 输出层
)

def init_weights(layer):
    if type(layer) == nn.Linear or type(layer) == nn.Conv2d:
        nn.init.xavier_uniform_(layer.weight)

model.apply(init_weights)
loss = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.9)

```

问题：

(1) 输出层 `Linear(84, 10)` 的输出维度为 10，对应 Fashion-MNIST 的几个类别？这 10 个类别分别是什么？

(2) 代码使用 `xavier_uniform_` 初始化权重，但**没有对 Sigmoid 激活层初始化**——这是正确的吗？为什么？

(3) 训练中使用的损失函数是 `CrossEntropyLoss`，它适用于分类还是回归任务？它在内部集成了哪个激活操作？

(4) 若将所有 `nn.Sigmoid()` 替换为 `nn.ReLU()`，从梯度消失的角度分析，这样修改对训练有什么好处？

来源溯源：[文档第 8 章 8.5 案例：服装分类]

小白必读：为什么这里用 Sigmoid 而不是 ReLU？

- **历史背景：** LeNet-5（1998 年）提出时，ReLU 还没被广泛使用，Sigmoid 是当时的标准激活函数。文档案例刻意保留了原始的 Sigmoid，以还原 LeNet-5 的历史面貌。
- **Sigmoid 的梯度消失问题：** Sigmoid 的输出范围是 $(0,1)$，其导数最大值仅为 0.25（在输入为 0 时取到），在输入绝对值较大时导数趋近于 0。多层 Sigmoid 连乘后，浅层梯度极小，参数更新极慢——这正是文档第 5 章讨论的梯度消失问题。
- **ReLU 的优势：** $\mathrm{ReLU}(x) = \max(0, x)$，在正区间的导数恒为 1，不会因链式乘法而指数级衰减，大幅缓解梯度消失，训练更快更稳定。
- **Xavier 初始化**：专门为 Sigmoid/Tanh 设计，通过控制初始权重的方差，使各层激活值的方差尽量一致，让梯度传播更平稳。对于 ReLU，则推荐使用 He 初始化（`kaiming_uniform_`）。

参考答案与详细解析

(1) 输出层与类别：

`Linear(84, 10)` 输出 **10** 维向量，对应 Fashion-MNIST 的 **10 个服装类别**：

标签	类别	标签	类别
0	T 恤/上衣	5	凉鞋
1	裤子	6	衬衫
2	套头衫	7	运动鞋
3	连衣裙	8	包
4	外套	9	靴子

(2) xavier_uniform_ 没有初始化激活层——正确！

`xavier_uniform_` 是对**权重矩阵**的初始化，而 Sigmoid（`nn.Sigmoid()`）、ReLU、池化层等**没有可学习的参数**（没有权重矩阵），因此无需初始化。`model.apply(init_weights)` 中的条件判断 `type(layer) == nn.Linear or type(layer) == nn.Conv2d` 已经精确地只初始化有权重的层，设计是正确的。

(3) CrossEntropyLoss 适用场景：

`CrossEntropyLoss` 适用于**多分类任务**（分类问题）。它在内部集成了 **Softmax** 操作——即先对网络最后一层的 logits（原始输出）做 Softmax 转化为概率分布，再计算交叉熵损失。因此使用 `CrossEntropyLoss` 时，最后一层（`Linear(84, 10)`）**不需要再手动加 Softmax**，否则会做两次 Softmax 导致梯度计算出错。

(4) Sigmoid 换为 ReLU 的好处：

- **缓解梯度消失**：ReLU 在正区间导数恒为 1，链式法则乘积不会趋近于 0；而 Sigmoid 导数最大仅 0.25，三层 Sigmoid 连乘后梯度最多剩 $0.25^3 \approx 1.5\%$。
- **训练速度更快**：梯度信号更强，参数更新幅度更大，收敛轮次减少。
- **代价**：若使用 ReLU，Xavier 初始化不再最优，应改用 **He 初始化**（`kaiming_uniform_`），以补偿 ReLU 将约一半输入置零带来的方差损失。

修改建议：现代化 LeNet-5

```python
def init_weights(layer):
    if type(layer) == nn.Linear or type(layer) == nn.Conv2d:
        # ReLU 配套使用 He 初始化
        nn.init.kaiming_uniform_(layer.weight, nonlinearity='relu')

model_modern = nn.Sequential(
    nn.Conv2d(1, 6, kernel_size=5, padding=2),
    nn.ReLU(),            # 替换 Sigmoid
    nn.AvgPool2d(2, 2),
    nn.Conv2d(6, 16, 5),
    nn.ReLU(),            # 替换 Sigmoid
    nn.AvgPool2d(2, 2),
    nn.Flatten(),
    nn.Linear(400, 120),
    nn.ReLU(),            # 替换 Sigmoid
    nn.Linear(120, 84),
    nn.ReLU(),            # 替换 Sigmoid
    nn.Linear(84, 10),    # 最后一层不加激活，CrossEntropyLoss 自带 Softmax
)

```

现实应用场景

为什么工业界几乎放弃了 Sigmoid（除了输出层）？

2012 年 AlexNet 横空出世时，其最大改动之一就是将网络中所有隐藏层的 Sigmoid 替换为 ReLU，并证明这一改动使训练速度提升了约 6 倍。此后，ReLU 及其变体（Leaky ReLU、ELU、GELU）成为深层网络中间层的主流激活函数，Sigmoid 仅保留在二**分类输出层**（将 logit 映射到 $[0,1]$ 概率）和**门控机制**（LSTM、注意力机制中的门控信号）中。现代大语言模型（如 GPT 系列）的 Transformer 块中使用的是 GELU（高斯误差线性单元），在 ReLU 的基础上引入了更平滑的梯度过渡，进一步提升了深层网络的训练稳定性。